UNIVERSIDAD TECNOLÓGICA DE LOS ANDES

FACULTAD DE INGENIERÍA

DEPARTAMENTO ACADÉMICO DE INGENIERÍAS

E.P. DE ING. DE SISTEMAS E INFORMÁTICA

Computación Gráfica

INGENIERO

Tintaya Zegarra Elvio

TAREA

"Investigar sobre Curvas Paramétricas y Mallas Poligonales"

ESTUDIANTE

Santos Segovia Jose Luis

CUSCO – PERÚ

2026 – I

Investigar sobre Curvas Paramétricas y Mallas Poligonales

Índice

I. Introducción 3

II. Curvas Paramétricas 4

2.1. Definición 4

2.2. Ejemplo clásico: circunferencia 4

2.3. Aplicaciones 9

2.4. Relación con el código del ejemplo 9

III. Mallas Poligonales 10

3.1. Definición 10

3.2. Componentes principales 10

3.3. Tipos de mallas 10

3.4. Aplicaciones 15

3.5. Relación con el código de ejemplo 16

IV. Importancia En La Computación Gráfica 16

V. Conclusiones 17

VI. Referencias 18

I. Introducción

En el campo de la computación gráfica, la representación y manipulación de formas geométricas constituyen la base para el desarrollo de aplicaciones visuales modernas, tales como videojuegos, simuladores, animaciones y entornos de realidad virtual. En este contexto, las **curvas paramétricas** y las **mallas poligonales** desempeñan un papel fundamental, ya que permiten modelar tanto trayectorias suaves como estructuras tridimensionales complejas de manera eficiente y precisa.

Por un lado, las curvas paramétricas ofrecen una forma flexible de describir movimientos y formas continuas mediante ecuaciones matemáticas dependientes de un parámetro, lo que facilita la generación de animaciones dinámicas y trayectorias controladas. Estas curvas son ampliamente utilizadas en el diseño gráfico, la tipografía digital y la simulación de movimientos naturales.

Por otro lado, las mallas poligonales constituyen la estructura básica para la representación de objetos en tres dimensiones, al estar formadas por vértices, aristas y caras que definen la geometría de un modelo. Su uso es predominante en el modelado 3D, permitiendo construir desde objetos simples hasta escenarios altamente detallados utilizados en diversas aplicaciones tecnológicas.

La presente investigación tiene como objetivo analizar los fundamentos teóricos de ambos conceptos, así como demostrar su aplicación práctica mediante ejemplos desarrollados en código. De esta manera, se busca no solo comprender su importancia dentro de la computación gráfica, sino también evidenciar cómo estos elementos se integran en la creación de representaciones visuales interactivas.

II. Curvas Paramétricas

2.1.Definición

Una curva paramétrica es una representación matemática en la que las coordenadas de los puntos de una curva se expresan como funciones de una variable independiente, llamada parámetro.

De acuerdo con James D. Foley (1996), una curva paramétrica se define como:

- $x = f(t)$
- $y = g(t)$
- $z = h(t)$ (en 3D)

Donde **t** es el parámetro que varía en un intervalo determinado.

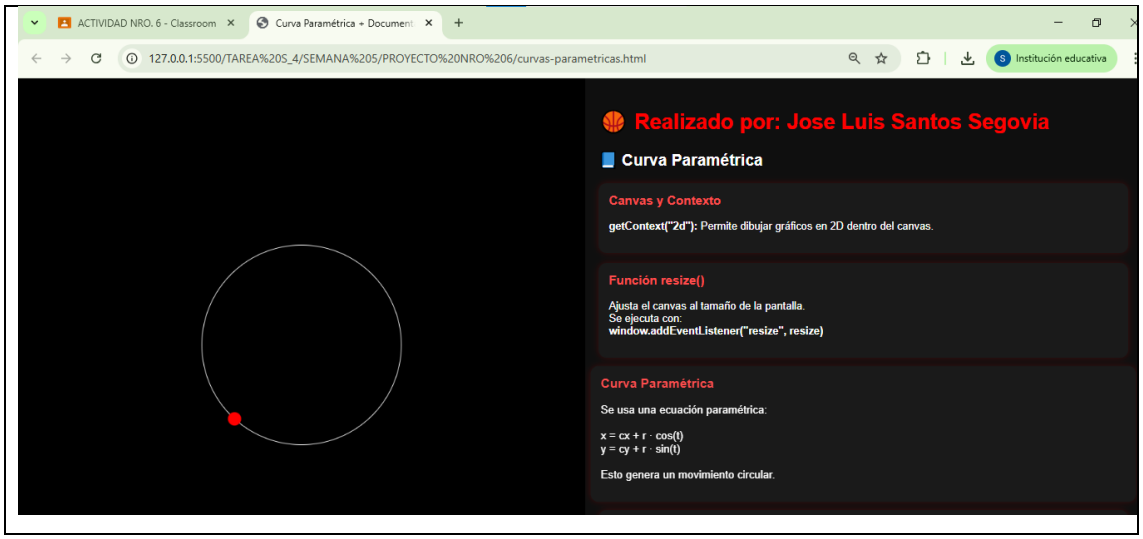
2.2.Ejemplo clásico: circunferencia

Una de las curvas paramétricas más utilizadas es la circunferencia:

- $x = r \cos(t)$
- $y = r \sin(t)$

Esta representación permite generar movimiento circular.

Ejemplo Aplicado:



Código:

```html
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Curva Paramétrica + Documentación</title>

<style>
body {
    margin: 0;
    display: flex;
    height: 100vh;
    background: black;
    color: white;
    font-family: Arial;
}

/* IZQUIERDA */
#contenedorCanvas {
    width: 50%;
}

/* DERECHA */
#documentacion {
    width: 50%;
    padding: 20px;
    background: #0f0f0f;
    overflow-y: auto;
}

/* TARJETAS */
.card {
```

```
    background: #1a1a1a;
    border-radius: 12px;
    padding: 15px;
    margin-bottom: 15px;
    box-shadow: 0 0 10px rgba(255,0,0,0.2);
    transition: transform 0.2s;
}

.card:hover {
    transform: scale(1.03);
}

.card h3 {
    margin-top: 0;
    color: #ff4d4d;
}

canvas {
    display: block;
}
</style>
</head>
<body>

<div id="contenedorCanvas">
    <canvas id="canvas"></canvas>
</div>

<div id="documentacion">
    <h1 style="color: red;">🏀 Realizado por: Jose Luis Santos
Segovia</h1>
    <h2>📄 Curva Paramétrica</h2>

    <div class="card">
        <h3>Canvas y Contexto</h3>
        <p>
        <b>getContext("2d"):</b> Permite dibujar gráficos en 2D dentro
del canvas.
        </p>
    </div>

    <div class="card">
        <h3>Función resize()</h3>
        <p>
        Ajusta el canvas al tamaño de la pantalla.<br>
        Se ejecuta con:<br>
        <b>window.addEventListener("resize", resize)</b>
        </p>
    </div>
```

```html
    <div class="card">
        <h3>Curva Paramétrica</h3>
        <p>
        Se usa una ecuación paramétrica:<br><br>
        x = cx + r · cos(t)<br>
        y = cy + r · sin(t)<br><br>
        Esto genera un movimiento circular.
        </p>
    </div>

    <div class="card">
        <h3>Centro y Radio</h3>
        <p>
        <b>cx, cy:</b> centro del canvas<br>
        <b>r:</b> radio de la circunferencia
        </p>
    </div>

    <div class="card">
        <h3>Dibujo de la órbita</h3>
        <p>
        <b>arc(cx, cy, r, 0, 2π):</b><br>
        Dibuja la circunferencia completa.
        </p>
    </div>

    <div class="card">
        <h3>Partícula en movimiento</h3>
        <p>
        Se dibuja un punto rojo que sigue la curva usando:<br>
        <b>arc(x, y, 10, 0, 2π)</b>
        </p>
    </div>

    <div class="card">
        <h3>Animación</h3>
        <p>
        <b>t += 0.02:</b> avanza el tiempo<br>
        <b>requestAnimationFrame():</b> actualiza la animación de forma
fluida
        </p>
    </div>

</div>

<script>
// Obtiene el elemento <canvas> del HTML
const canvas = document.getElementById("canvas");
```

```
// Obtiene el contexto 2D del canvas (donde se dibuja)
const ctx = canvas.getContext("2d");

// Función para ajustar el tamaño del canvas
function resize() {
    // Ancho será la mitad de la pantalla
    canvas.width = window.innerWidth / 2;

    // Alto será toda la altura de la ventana
    canvas.height = window.innerHeight;
}

// Detecta cuando se redimensiona la ventana
window.addEventListener("resize", resize);

// Llama a la función al inicio para establecer tamaño inicial
resize();

// Variable de tiempo (ángulo en radianes)
let t = 0;

// Función principal de animación
function animar() {

    // Limpia todo el canvas en cada frame (para evitar trazos
acumulados)
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // Coordenadas del centro del canvas
    let cx = canvas.width / 2;
    let cy = canvas.height / 2;

    // Radio de la órbita
    let r = 150;

    // Calcula la posición X usando coseno (movimiento circular)
    let x = cx + r * Math.cos(t);

    // Calcula la posición Y usando seno (movimiento circular)
    let y = cy + r * Math.sin(t);

    // ========================
    // DIBUJAR ÓRBITA (CÍRCULO)
    // ========================
    ctx.beginPath(); // Inicia un nuevo dibujo
    ctx.strokeStyle = "white"; // Color del borde
    ctx.arc(cx, cy, r, 0, Math.PI * 2); // Dibuja un círculo completo
    ctx.stroke(); // Muestra el contorno
```

```
    // ========================
    // DIBUJAR PARTÍCULA
    // ========================
    ctx.beginPath(); // Nuevo dibujo
    ctx.fillStyle = "red"; // Color de relleno
    ctx.arc(x, y, 10, 0, Math.PI * 2); // Círculo pequeño (la
partícula)
    ctx.fill(); // Rellena el círculo

    // Incrementa el valor del tiempo para que se mueva
    t += 0.02;

    // Llama nuevamente a la función para crear animación fluida
    requestAnimationFrame(animar);
}

// Inicia la animación
animar();
</script>

</body>
</html>
```

2.3.Aplicaciones

Las curvas paramétricas se utilizan en:

- Animaciones (movimiento de objetos)
- Diseño de trayectorias
- Modelado de superficies
- Tipografía digital (curvas Bézier)

Según David F. Rogers (2001), estas curvas son esenciales para representar formas suaves y continuas en gráficos por computadora.

2.4.Relación con el código del ejemplo

Mi implementación:

```
x = cx + r * Math.cos(t);
y = cy + r * Math.sin(t);
```

- Representa una curva paramétrica circular
- Usa el parámetro **t como tiempo**
- Aplica animación con requestAnimationFrame

Esto es exactamente un caso real de aplicación en computación gráfica.

III. Mallas Poligonales

3.1. Definición

Una malla poligonal es una estructura compuesta por vértices, aristas y caras que define la forma de un objeto en el espacio.

Según Alan Watt (2000), las mallas poligonales son la forma más común de representar objetos tridimensionales en gráficos computacionales.

3.2. Componentes principales

- **Vértices**: puntos en el espacio
- **Aristas**: líneas que conectan vértices
- **Caras**: polígonos (generalmente triángulos o cuadrados)

3.3. Tipos de mallas

- Mallas de triángulos (las más usadas en gráficos 3D)
- Mallas cuadrangulares
- Mallas irregulares

Según Peter Shirley (2015), los triángulos son preferidos porque siempre son planos y fáciles de procesar.

Ejemplo aplicado:

Código:

```html
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Malla Poligonal + Documentación</title>

<style>
body {
    margin: 0;
    display: flex;
    height: 100vh;
    background: black;
    color: white;
    font-family: Arial;
}

/* IZQUIERDA */
#contenedorCanvas {
    width: 50%;
}

/* DERECHA */
#documentacion {
    width: 50%;
    padding: 20px;
    background: #0f0f0f;
    overflow-y: auto;
}

/* TARJETAS */
```

```
.card {
    background: #1a1a1a;
    border-radius: 12px;
    padding: 15px;
    margin-bottom: 15px;
    box-shadow: 0 0 10px rgba(0,255,255,0.2);
    transition: transform 0.2s;
}

.card:hover {
    transform: scale(1.03);
}

.card h3 {
    margin-top: 0;
    color: cyan;
}

canvas {
    display: block;
}
</style>
</head>
<body>

<div id="contenedorCanvas">
    <canvas id="canvas"></canvas>
</div>

<div id="documentacion">
    <h1 style="color: cyan;">🏀 Realizado por: Jose Luis Santos
Segovia</h1>
    <h2>▨ Mallas Poligonales</h2>

    <div class="card">
        <h3>Canvas y Contexto</h3>
        <p><b>getContext("2d"):</b> Permite dibujar gráficos en 2D
dentro del canvas.</p>
    </div>

    <div class="card">
        <h3>Función resize()</h3>
        <p>
        Ajusta el tamaño del canvas al tamaño de la ventana.<br>
        Se ejecuta con:<br>
        <b>window.addEventListener("resize", resize)</b>
        </p>
    </div>
```

```
    <div class="card">
        <h3>Animación</h3>
        <p>
        <b>requestAnimationFrame(animar):</b><br>
        Permite ejecutar la animación de forma fluida y continua.
        </p>
    </div>

    <div class="card">
        <h3>Centrado de la Malla</h3>
        <p>
        ancho = columnas × tamaño<br>
        alto = filas × tamaño<br><br>
        Se centra usando:<br>
        (canvas - tamañoMalla) / 2
        </p>
    </div>

    <div class="card">
        <h3>Dibujado de Polígonos</h3>
        <p>
        <b>beginPath()</b> → inicia figura<br>
        <b>moveTo()</b> → punto inicial<br>
        <b>lineTo()</b> → dibuja líneas<br>
        <b>closePath()</b> → cierra figura<br>
        <b>stroke()</b> → dibuja contorno
        </p>
    </div>

    <div class="card">
        <h3>Movimiento Ondulado</h3>
        <p>
        <b>Math.sin()</b> genera el efecto de onda.<br>
        Se combina con el tiempo para animación continua.
        </p>
    </div>

</div>

<script>
// Obtiene el elemento canvas del HTML
const canvas = document.getElementById("canvas");

// Obtiene el contexto 2D donde se dibujará
const ctx = canvas.getContext("2d");

// Función para ajustar el tamaño del canvas
function resize() {
    // Ancho: la mitad de la pantalla
```

```
    canvas.width = window.innerWidth / 2;

    // Alto: toda la altura de la ventana
    canvas.height = window.innerHeight;
}

// Evento que detecta cuando se cambia el tamaño de la ventana
window.addEventListener("resize", resize);

// Llama la función al inicio
resize();

// Número de filas de la malla
let filas = 20;

// Número de columnas de la malla
let columnas = 20;

// Tamaño de cada celda (cuadrado)
let tamaño = 30;

// Variable de tiempo para animación
let tiempo = 0;

// Función principal de animación
function animar() {

    // Limpia el canvas en cada frame
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // Calcula el ancho total de la malla
    let anchoMalla = columnas * tamaño;

    // Calcula el alto total de la malla
    let altoMalla = filas * tamaño;

    // Calcula la posición inicial en X para centrar la malla
    let inicioX = (canvas.width - anchoMalla) / 2;

    // Calcula la posición inicial en Y para centrar la malla
    let inicioY = (canvas.height - altoMalla) / 2;

    // Recorre las filas
    for (let i = 0; i < filas; i++) {

        // Recorre las columnas
        for (let j = 0; j < columnas; j++) {

            // Posición X de cada celda
```

```
            let x = inicioX + j * tamaño;

            // Posición Y de cada celda
            let y = inicioY + i * tamaño;

            // Offset vertical usando función seno (crea efecto de
onda)
            let offset = Math.sin((i + j + tiempo) * 0.3) * 10;

            // ========================
            // DIBUJAR CUADRADO
            // ========================
            ctx.beginPath(); // Inicia nuevo trazo
            ctx.strokeStyle = "cyan"; // Color del borde

            // Dibuja los 4 lados del cuadrado con desplazamiento
(offset)
            ctx.moveTo(x, y + offset); // Punto superior izquierdo
            ctx.lineTo(x + tamaño, y + offset); // superior derecho
            ctx.lineTo(x + tamaño, y + tamaño + offset); // inferior
derecho
            ctx.lineTo(x, y + tamaño + offset); // inferior izquierdo

            ctx.closePath(); // Cierra la figura
            ctx.stroke(); // Dibuja el contorno
        }
    }

    // Incrementa el tiempo para animar la onda
    tiempo += 0.05;

    // Llama de nuevo la función para animación continua
    requestAnimationFrame(animar);
}

// Inicia la animación
animar();
</script>

</body>
</html>
```

3.4.Aplicaciones

Las mallas poligonales se usan en:

- Videojuegos
- Modelado 3D
- Realidad virtual
- Simulación física

3.5.Relación con el código de ejemplo

El código implementa una malla 2D basada en cuadrados:

- Uso de filas y columnas
- Generación de polígonos (cuadrados)
- Animación con función seno

Ejemplo clave:

```
let offset = Math.sin((i + j + tiempo) * 0.3) * 10;
```

Esto simula una **deformación dinámica de la malla**, similar a efectos usados en simulaciones de agua o superficies.

IV. Importancia En La Computación Gráfica

Ambos conceptos son fundamentales porque:

- Las **curvas paramétricas** permiten definir trayectorias y formas suaves
- Las **mallas poligonales** permiten construir objetos complejos

Por otro lado, según Donald Hearn (2011), la combinación de estas técnicas es clave para el desarrollo de gráficos modernos.

V. Conclusiones

- Las curvas paramétricas constituyen una herramienta fundamental en la computación gráfica, ya que permiten representar trayectorias y formas continuas mediante funciones matemáticas, facilitando la creación de animaciones suaves y controladas.
- La implementación práctica de curvas paramétricas, como la circunferencia mediante funciones seno y coseno, demuestra su aplicabilidad directa en el desarrollo de gráficos dinámicos, evidenciando la relación entre teoría matemática y programación.
- Las mallas poligonales son esenciales para la representación de objetos en entornos digitales, debido a que permiten construir estructuras tridimensionales a partir de elementos básicos como vértices, aristas y caras.
- El uso de mallas, especialmente basadas en polígonos simples como triángulos y cuadrados, facilita el procesamiento gráfico y la simulación de efectos visuales, como deformaciones y movimientos ondulatorios, ampliamente utilizados en simulaciones y videojuegos.
- La combinación de curvas paramétricas y mallas poligonales resulta clave en la computación gráfica moderna, ya que permite integrar movimiento y estructura, siendo la base para el desarrollo de aplicaciones avanzadas en modelado 3D, animación y realidad virtual.

VI. Referencias

- Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1996). Computer Graphics: Principles and Practice. Addison-Wesley. https://cgisutic.wordpress.com/wp-content/uploads/2016/07/computer_graphics_principles_and_practice.pdf
- Hearn, D., & Baker, M. P. (2011). Computer Graphics with OpenGL. Pearson. https://ingenieriayeducacion.wordpress.com/wp-content/uploads/2013/12/graficosporcomputadorayopengl.pdf
- Rogers, D. F. (2001). Mathematical Elements for Computer Graphics. McGraw-Hill. https://es.scribd.com/document/515029974/Pdfcoffee-com-David-f-Rogers-j-Alan-Adams-Mathematical-Elements-for-Computer-Graphics-1989-Mcgraw-Hillpdf-PDF-Free
- Shirley, P. (2015). Fundamentals of Computer Graphics. CRC Press. https://repo.darmajaya.ac.id/5422/1/Fundamentals%20of%20Computer%20Graphics%2C%20Fourth%20Edition%20%28%20PDFDrive%20%29.pdf
- Watt, A. (2000). 3D Computer Graphics. Addison-Wesley. https://archive.org/details/3DComputerGraphicsAlanWatt3thEditionSelectedChapters/page/n25/mode/2up